

The Signet

Security Brief – Firmware v1.3.0+

Version 1.3.0 firmware represents the most security-hardened release to date, incorporating lessons from two independent audits and real-world field testing. Every design decision prioritizes the operator's safety, anonymity, and plausible deniability.

Zero Persistence Architecture

The Signet writes zero bytes to permanent storage during normal operation. Messages exist only in volatile RAM and are irrecoverably destroyed the instant the device loses power. There is no message history, no activity log, no usage record, and no settings file written to flash —ever.

Previous firmware versions stored a two-byte language preference in non-volatile storage. Version 1.3.0 eliminates even this, achieving a fully stateless design. The language selection modal appears fresh on every boot, and the chosen language lives only in RAM for the duration of that session. If the device is seized, powered off, or its battery dies, there is nothing to recover. Flash forensics will find only the firmware binary itself and static UI assets (fonts, images). No user-generated data touches persistent memory under any circumstances.

No Telemetry. No Logs. No Exceptions.

The Signet firmware contains zero outbound network code. There is no HTTP client library, no WiFi client mode, no DNS lookup to external servers, no cloud dependency, no analytics SDK, no crash reporter, and no phone-home mechanism of any kind. This has been verified through static analysis of every `#include` directive and every function call in the codebase.

The device operates exclusively as a WiFi Access Point. It creates a local network that allows a single connection; it never joins one. It serves a web UI to the connected client; it never contacts anything beyond that single direct connection. There are no beacon frames sent to probe for known networks, no mDNS broadcasts, and no service discovery announcements. The only RF emission is the AP's SSID beacon, and even that is time-limited (see below).

MAC Address Locking — Single Operator Exclusivity

When the first device connects to The Signet's WiFi access point and issues any API command, the firmware locks to that device's MAC address. From that point forward, no other device can control The Signet — even if it somehow connects to the WiFi network. The lock persists until the device is power-cycled.

This is enforced at the firmware level through `esp_wifi_ap_get_sta_list()`, comparing the requesting client's MAC against the locked MAC on every API call. Combined with the ESP32's `max_conn=1` parameter (which limits the AP to a single simultaneous WiFi client), this creates a two-layer access control: only one device can connect, and only the first device to authenticate gets command authority.

Automatic AP Shutdown on Disconnect

Version 1.3.0 introduces a critical improvement: when the locked operator disconnects from WiFi, the access point shuts down entirely. The firmware polls the station list each loop iteration, and the moment the locked client's MAC disappears, it executes a full shutdown sequence — DNS server stopped, `WiFi.softAPdisconnect(true)` called, WiFi radio turned off. No second device can connect during any window between disconnect and shutdown because the check runs continuously with no grace period.

After shutdown, the locked MAC address is scrubbed from RAM with `memset(lockedMac, 0, sizeof(lockedMac))`. This is a deliberate anti-forensics measure: even a cold-boot attack or live memory dump after the operator walks away will not reveal which device was controlling The Signet. Morse playback continues uninterrupted — the beacon keeps transmitting its message even with WiFi completely disabled.

OTA Combination Lock — Physical Verification for Firmware Updates

Over-the-air firmware updates are protected by a hardware-verified combination lock system. Before any firmware binary can be uploaded, the operator must:

- 1. Request a code** — The device generates a random 3-digit code (each digit 1–9) using the ESP32's hardware random number generator (`esp_random()`).
- 2. Read the code from the LED** — The RGB LED blinks the code physically: red blinks for digit one, green blinks for digit two, blue blinks for digit three. For example, 3 red blinks, 7 green blinks, 2 blue blinks = code 372.
- 3. Enter the code in the web UI** — The operator types the three digits into color-coded input fields.
- 4. Upload the firmware** — Only after successful code verification does the upload endpoint accept the binary.

The code is generated fresh each time and invalidated after a single successful use. This means an attacker cannot replay a previously observed code, and they cannot upload firmware without physical line-of-sight to the device's LED. The combination lock transforms OTA security from a pure software problem into a physical presence requirement.

Additionally, the firmware validates uploaded binaries against the ESP32 magic byte (0xE9), rejecting anything that isn't a legitimate ESP32 firmware image. The device uses a dual-partition OTA layout, enabling automatic rollback to the previous working firmware if an update fails mid-write.

60-Second WiFi Exposure Window

The Signet's WiFi access point automatically shuts down after 60 seconds of inactivity. This minimizes the window during which the device's SSID is visible to passive WiFi scanners. The operator connects, configures their message, and the AP disappears — all within a brief window. The Morse beacon continues transmitting indefinitely after WiFi shutdown, operating in complete radio silence on the WiFi band.

This is a deliberate privacy trade-off: the unique SSID (which includes a MAC-derived suffix for operator identification of their own device) is only broadcast during the brief configuration period, not during the potentially hours-long transmission phase.

Randomized WiFi Channel Selection

Each time The Signet boots, it randomly selects one of the three non-overlapping 2.4 GHz channels (1, 6, or 11) using the ESP32's hardware RNG. This provides automatic interference avoidance in congested RF environments and adds a minor layer of unpredictability — an observer cannot assume the device will always appear on the same channel between power cycles.

Defense-in-Depth Input Validation

Every API endpoint enforces strict input validation and payload size limits as a defense against malformed or malicious requests:

- Payload size caps:** `/api/update`` and `/api/play`` reject any request body exceeding 512 bytes (HTTP 413). `/api/language`` caps at 256 bytes. These limits prevent buffer-based attacks and flash wear from oversized payloads.
- RGB value constraining:** All color values are clamped to 0–255 using `constrain()`, preventing out-of-range writes.
- WPM bounds enforcement:** Speed values are validated to the 5–20 WPM range before use, with a division-by-zero guard in the timing calculation.
- Language index validation:** Language codes are bounds-checked against the valid range before any array access.
- Firmware header validation:** OTA uploads are checked for the ESP32 magic byte (0xE9) before the write begins.

No App Required — Captive Portal Web UI

The Signet requires no app installation, no account creation, no download, and no third-party dependency. The operator connects to the WiFi AP, and the device's captive portal detection triggers the web UI automatically. Version 1.3.0+ includes explicit handlers for every major platform's captive portal probe mechanism — Android, iOS, Windows 10/11, macOS, and Firefox — ensuring the UI appears reliably across devices without requiring the operator to manually navigate to an IP address.

This matters for security because there is no app binary that could be discovered on a seized phone, no account that could be subpoenaed, and no download history that could be forensically recovered. The entire interaction happens through a standard web browser session that leaves minimal trace.

Multi-Language Support Without Fingerprinting

The Signet supports eight languages — English, Spanish, French, Russian, Simplified Chinese, Traditional Chinese, Farsi, and Arabic — with full right-to-left layout support for Arabic and Farsi. In v1.3.0, the language preference is never written to flash, eliminating it as a potential demographic fingerprint. The selection modal appears on every boot, and the choice is stored only in volatile RAM for that session.

dependencies (ArduinoJson, FastLED). Anyone can audit the code, verify the privacy claims, compile their own build, or fork the project entirely.

Thread Safety & Watchdog Protection

The firmware uses FreeRTOS primitives to ensure safe concurrent operation between the web server (running on the main loop) and the Morse playback task (running on a dedicated core):

- **Mutex-protected text access:** The message string is guarded by a FreeRTOS semaphore. Both read and write operations acquire the mutex with a 100ms timeout, with a safe fallback to "SOS" if the lock cannot be obtained.
- **Atomic playback state:** The `playing` flag uses `std::atomic<bool>` for lock-free, thread-safe reads and writes across tasks.
- **Fatal mutex failure:** If either the text mutex or OTA mutex fails to initialize at boot, the firmware halts execution entirely rather than running in an unsafe state.
- **Task Watchdog Timer:** A 60-second hardware watchdog timer automatically resets the device if the main loop hangs, protecting against denial-of-service conditions or firmware bugs that could leave the device unresponsive.

Open Source Transparency

The Signet's firmware is fully open source. The complete `.ino` source code, schematics, and bill of materials are available for inspection. There are no binary blobs, no obfuscated modules, and no proprietary libraries beyond the standard ESP32 SDK and well-audited open-source dependencies (ArduinoJson, FastLED). Anyone can audit the code, verify the privacy claims, compile their own build, or fork the project entirely.

The device also ships unlocked by default — the operator can upload their own firmware at any time. The combination lock protects against unauthorized uploads, not against the owner's right to modify their own hardware.

The Signet

BIG BROTHER IS WATCHING YOU.... Send him a message.

quiklearner2099@proton.me

<https://www.tindie.com/products/signet-systems/the-signet/>

 Hand Made with Pride in Canada 🇨🇦